

העולם המגניב

שקד

ספטמבר 17, 2026 (ו' בתשרי ה'תשפ"ז)

תוכן העניינים

שפת אסמבלי, בניגוד ל- C , ייחודית לכל מעבד. אנחנו נלמד סט פקודות המתאימות למעבד מסויים. לרוב, למעבדים של אותה חברה יהיו את אותן פקודות. כל מעבד מגיע עם אסמבלר מובנה מראש שמתרגם את שפת האסמבלי שהוא קורא(עם הפקודות הייחודיות לו) לשפת מכונה.

מעבד שתומך ברשימת פקודות בעצם מיישם סטנדרט שנקרא ISA שזה. בתחילת עולם המחשוב השתמשו במשהו שנקרא $CISC$ שאומר שככל שמתקדמים נוסף עוד ועוד פקודות כדי שהמעבד יוכל לעשות דברים יותר מורכבים.

כיום משתמשים לרוב ב- $RISC$ (שאתו גם אנחנו נעבוד) שהוא נותן סט פקודות קטן אשר מאפשרות לך לבצע כל דבר **ואתה** הוא שתתכנת את התוכניות המסובכות באמצעות מעט הפקודות הפשוטות. כל הפקודות נמצאות על יצור בשם **הכרטיס הירוק** (התמונה במצגת לא באיכות טובה ולא הצלחתי למצוא אותה באינטרנט - לטענת המצגת יש אותה במודל אבל לא הצלחתי למצוא אותה במודל).

1 הרגיסטרים

בניגוד לשפות תכנות עיליות כמו $Java$ או C , באסמבלי אין לנו משתנים כפי שהכרנו אותם. יש לנו **רגיסטרים** שהם מספר סופי ומוגבל של מקומות מיוחדים שמחזיקים ערך יחיד. הם בנויים ישירות לתוך החומרה. פעולות יכולות להתבצע רק על מידע המאוחסן ברגיסטרים ולכן הם מאוד חשובים לנו, גם בתור מדמחיסטים וגם בתור חשמליסטים. הייתרון של הרגיסטרים הוא שהם ממש קרובים למעבד ולכן **מאוד** מהירים. בריסק-5 יש 32 רגיסטרים הנקראים $x0 - x31$ אך לנו יש רק 31 רגיסטרים לעבוד איתם שכן רגיסטר $x0$ הוא מיוחד. לכן, נדרש להיות **מאוד** יעילים עם המידע שלנו כדי שלא נעבור את מספר הרגיסטרים הקיים שכן פשוטו של עניין - אין עוד רגיסטרים.

הגדרה 1.1 (מילה). קבוצה של 32 ביטים נקראת **מילה** בריסק-5.

הערה. זו לא כמו מילה של מתקא):

הערה (אמיתית הפעם). ניתן לקרוא רק $4GB$ (4 גיגה בית) עם מילה של ריסק-5 מכיוון שכדי להגיע לשם צריך לכתוב את הערך שאנחנו צריכים ברגיסטר וגם אם נמלא את כל הסיביות באחדים נקבל 2^{32} שזה 4 גיגה.

בניגוד לשפות תכנות עיליות, אין סוגי משתנים ברגיסטרים וכל רגיסטר יכול להכיל כל

דבר ואנו צריכים לזכור מה שמנו בכל רגיסטר.

Register	Name ABI	Description	Saver
$x0$	<i>zero</i>	zero Hard-wired	—
$x1$	<i>ra</i>	address Return	Caller
$x2$	<i>sp</i>	pointer Stack	Callee
$x3$	<i>gp</i>	pointer Global	—
$x4$	<i>tp</i>	pointer Thread	—
$x5$	t_0	register link Temporary/alternative	Caller
$x6 - 7$	t_{1-2}	Temporaries	Caller
$x8$	s_0/fp	pointer register/frame Saved	Callee
$x9$	$s1$	register Saved	Callee
$x10 - 11$	a_{0-1}	values arguments/return Function	Caller
$x12 - 17$	a_{2-7}	arguments Function	Caller
$x18 - 27$	s_{2-11}	registers Saved	Callee
$x28 - 31$	t_{3-6}	Temporaries	Caller

2 פקודות ריסק-5

לכל פקודה יש *opcode* שהוא המזהה של הפקודה ו-*operands* שהם מה שהפקודה מקבלת(כמו פונקציה ב-C)

```
1 add x1, x2, x3 # x1 = >x2< + >x3<
```

בואו ננתח את השורה הזו:

- מתחילים עם שם הפקודה, ה-*opcode* שבמקרה הזה הוא . זוהי פקודה שמוסיפה.
- אחרי זה מופיע רגיסטר היעד שאליו נכנס הערך המחושב (כמעט תמיד רגיסטר היעד הוא האופרנד הראשון שנכתוב)
- לאחריו יש את שני האופרנדים החשובים: המחובר הראשון והמחובר השני.
- לאחר מכן יש # שהוא הערה ומה שנכתב לאחריו לא מחושב (כמו // ב-C)

והנה רשימת הפקודות:

2.1 פקודות בסיסיות

- מוסיפה את $x2$ ו- $x3$ ושמה את הערך ב- $x1$.
- מחסרת את $x5$ מ- $x4$ ומציבה את הערך ב- $x3$.

דוגמה 2.1. עבור קוד ה-C:

```
1 a=b+c+d-e;
```

נכתוב באסמבלי של ריסק-5:

```
1 add x10, x1, x2 # temp = b + c
2 add x10, x10, x3 # temp = temp + d
3 sub x10, x10, x4 # a = temp - e
```

הערה. אם נרצה לשים ערך של רגיסטר אחד ברגיסטר אחר נשתמש ברגיסטר $x0$ שהוא תמיד אפס כך:

```
1 add x1, x3, x0
```

ואז זה ישים את הערך של $x3$ ברגיסטר $x1$ (כמובן שהערך ב- $x3$ לא משתנה)

- הפקודה הזו מחברת את הערך ב- $x4$ לקבוע -10 ושמה את הערך ב- $x3$. הפקודה תמיד תפעל עם רגיסטר יעד שלאחריו רגיסטר מקור ולאחריו ערך קבוע.

2.2 פקודות זיכרון

כעת אנו יכולים לבצע פעולות בין רגיסטרים, אך יש לנו רק מעט מהם ונדרש לשמור בזיכרון רחוק יותר ואיטי יותר ערכים שנרצה לשמור לאורך זמן. פקודות הפנייה לזיכרון הן איטיות ונביט בהן כעת.

אנחנו מתייחסים לכתובות של הזיכרון בקבוצות של 8 ביטים (שזה בית (*byte*) אחד) - ניתן לדמיין את הזיכרון כווקטור חד מימדי ארוך של מטריצות של 1×8 .

סיבית 1	ביט	<i>Bit (b)</i>
8 סיביות	בית	<i>Byte (B)</i>
16 סיביות	חצי-מילה	<i>Half word</i>
32 סיביות	מילה	<i>Word</i>

כדי לכתוב לזיכרון משהו באורך של יותר מביט, נכתוב תמיד לביט הכי פחות חשוב (*LSB*) - הימני ביותר.

תמיד מידע עובר מהזיכרון לרגיסטר או מהרגיסטר לזיכרון
אנו לא יכולים לעביר מידע ישר למעבד או ישר לזיכרון מהמעבד

- מה זה $12(x13)$ המוזר הזה? $x13$ מייצג את הרגיסטר הבסיסי שמצביע ל- $A[0]$ ו-12 זה מספר הבתים שאנחנו זזים כדי לקחת את $A[3]$. אזי, בעצם אנחנו לוקחים את הערך הנמצא ב- $A[3]$ ושמים אותו ברגיסטר $x10$.

- מכיוון שכל איבר במערך הוא 4 בתים, נכפיל 4 ב-10 כדי להגיע מ- $x13$ שמאחסן פוינטר לתחילת המערך, למיקום במערך שאנחנו רוצים.
 - כשמעתיקים בית מהזיכרון אל 32 בתים של רגיסטר, מתבצע תהליך של הרחבת הסימן על מנת שהסימן ישאר זהה, נניח אם הסימן הוא 0 למספר חיובי, כל שאר הסיביות של הרגיסטר יהיו אפסים ואז בסוף יופיע המספר.
 - פקודה לשמירה של בית יחיד בזיכרון sb עובדת גם היא באותו האופן שעבדה sw כך:
 - יש בנוסף פקודה לטעינת בית $unsigned$ - כלומר, ללא סימן. הפקודה היא lbu והיא מתנהגת באופן זהה ל- lw רק שאז הסימן לא ישנה.
 - **אין פקודה לשמירת בית $unsigned$ שכן אין לזה משמעות.**
- דוגמה 2.2.** מה יקרה כשנבצע את הקוד הזה:

```

1 addi x11,x0,0x3f5
2 sw x11,0)x5(
3 lb x12,1)x5(

```

תחילה, אנחנו שמים ברגיסטר $x11$ את הערך ההקסה $3f5$ ששווה ל-0011, 1111, 0101 בבינארי.

כעת, אנחנו שומרים את הערך שיש ב- $x11$ (כלומר את $3f5$ בהקסה) במיקום $x5$ בזיכרון.

כעת, אנו טוענים לרגיסטר $x12$ את הערך שיש בבית השני של $x5$ - כלומר, אנחנו טוענים את 0000, 0011 שהוא המספר 3 בבסיס עשרוני ובבסיס הקסה.

הערה. אם נרצה לתאר מספר בבסיס עשרוני נכתוב $0d$ לפניו.

אם נרצה לתאר מספר בבסיס הקסה נכתוב $0x$ לפניו.

אם נרצה לתאר מספר בבסיס בינארי נכתוב $0b$ לפניו.

2.3 פקודות לוגיות

operations Logical	operators C	operations RISC-V
AND Bit-by-bit	&	<i>and</i>
OR Bit-by-bit		<i>or</i>
XOR Bit-by-bit	^	<i>xor</i>
logical left Shift	<<	<i>sll</i>
logical right Shift	>>	<i>srl</i>

ניתן להשתמש בהן כך:

```
1 opcode >result <,>compare1 <,>compare2 <
```

הפעולה הלוגית נעשית על שני האופרנדנים ומושמת לרגיסטר הנמצא ב-*result*. לפקודות ההזזה, כמובן שהערך מגיע מרגיסטר אחר להזזה ומושם אחרי ההזזה ברגיסטר התוצאה. בנוסף, פקודות נוספות הינן:

- פקודת ההזזה שמאלה עם קבוע כך: (מכפילה ב- 2^k)
- פקודת הזזה ימינה עם קבוע כך: (מחלקת ב- 2^k)
- פקודת הזזה ימינה של המספר עם תיקון למען שמירת הסימן:
הערה. הפקודה *srai* היא לא חלוקה ב- 2^k .
זה עובד לרוב, אך לא עובד עם מספרים שליליים אי-זוגיים.
הערה. הפקודות *srai* ו-*srli* זהות במספרים חיוביים.

2.4 בקרת זרימה

כעת, נרצה משהו הדומה ל-*if* אחרת לא נוכל לכתוב תוכניות טובות. בנוסף, נרצה דרך לעבור ממקום למקום בשביל שנוכל לעשות לולאות בנוסף לדברים אחרים. הפקודה הראשונה שאנו נראה היא *beq* שמשווה בין שני רגיסטרים ואם הם שווים קופצת להצהרה בשם שכתבנו כך:

```
1 beq register1 , register2 , L1
```

ישנן עוד פקודות שעובדות כך והן:

- הפקודה *beq* (שראינו הרגע) משווה בין שני רגיסטרים ואם הם שווים קופצת להצהרה הרשומה.
- הפקודה *bne* משווה בין שני רגיסטרים ואם הם **לא שווים** קופצת להצהרה הרשומה.
- הפקודה *Blt* משווה בין שני רגיסטרים ואם הרגיסטר הראשון **קטן** מהרגיסטר השני אז היא קופצת להצהרה הרשומה.
- הפקודה *bge* משווה בין שני רגיסטרים ואם הרגיסטר הראשון **גדול או שווה** לרגיסטר השני אזי היא קופצת להצהרה הרשומה.
- הפקודה *J* קופצת תמיד להצהרה הרשומה (היא מקבלת רק הצהרה)

בואו נראה דוגמות:

דוגמה 2.3 (תוכנית *if* פשוטה). נניח התוכנית:

```

1  if ) i == j (
2      f = g + h;

```

ב-C תראה כך באסמבלי:

```

1  bne x13,x14,Exit
2  add x10,x11,x12
3
4  Exit:

```

דוגמה 2.4 (תוכנית *if – else* פשוטה). התוכנית:

```

1  if ) i == j (
2      f = g + h;
3  else
4      f = g - h;

```

ב-C תראה כך באסמבלי:

```

1  bne x13,x14, Else
2  add x10,x11,x12
3  j Exit
4
5  Else: sub x10,x11,x12
6  Exit:

```

דוגמה 2.5 (לולאה). נניח קוד ה-C הבא שמכיל לולאה:

```

1  int A]20[;
2  int sum = 0;
3  for )int i = 0; i > 20; i++(
4      sum += A]i[;

```

תראה כך באסמבלי כאשר $x8$ הוא הרגיסטר שמצביע על תחילת המערך:

```

1  add x9,x8,x0    # x9=&]0[
2  add x10,x0,x0   # sum=0
3  add x11,x0,x0   # i=0
4  addi x13,x0,20  # x13=20
5  Loop:
6      lw x12,0)(x9(    # x12=A]i[
7      add x10,x10,x12  # sum+=A]i[

```

```

8  addi x9,x9,4      # &A]++i[
9  addi x11,x11,1    # i++
10 blt  x11,x13,Loop

```

חלק I

קריאות לפונקציות

עוד לפני שנדבר על איך אנו יוצרים פונקציות, נדבר על איך משתמשים בהן. יש לנו שישה שלבים בקריאה לפונקציה:

1. נשים את הארגומנטים במקום שהפונקציה יכולה לגשת אליו.
 2. נעביר שליטה לפונקציה
 3. נאסוף את כל הזכרון שנדרש עבור הפונקציה
 4. נבצע את הפעולה של הפונקציה
 5. נשים את התוצאה איפה שהקוד שקרא יכול לגשת אליה ושחזור הרגיסטרים שהשתמשו בהם בפונקציה.
 6. החזרת שליטה למקור.
- כדי לשמור את המיקום אליו צריך לחזור לאחר הפונקציה, נשתמש ברגיסטר *ra* אשר הינו קיצור ל-*address return*. בסוף פונקציה תמיד נבצע (הפקודה *jr* היא פסודו פקודה אשר מאפשרת קפיצה למיקום המאוחסן ברגיסטר המסויים שנקרא, במקרה הזה *ra*)
- דוגמה 2.6.** נממש את הפונקציה:

```

1 int sum(int x, int y)
2 {
3     return x + y;
4 }

```

וקריאה אליה כך:

```

1 1000 mv a0, s0
2 1004 mv a1, s1
3 1008 addi ra, zero, 1016

```

```

4 1012 j sum
5 1016 ...
6 ...
7 2000 sum: add a0, a0, a1
8 2004 jr ra

```

יש לנו פקודה שימושית על מנת לשמור את ה-*ra* ולקרוא לפונקציה באותו הזמן והיא *jal*. היא שומרת ב-*ra* את הכתובת הבאה מהשורה שהיא נרשמה בה וקוראת לפונקציה הרשומה.

דוגמה 2.7. בדוגמה שראינו קודם נוכל לכתוב:

```

1 1000 mv a0, s0
2 1004 mv a1, s1
3 1008 jal sum
4 ....

```

ישנו בנוסף, קיצור עבור *jr ra* עם הרגיסטר *ra* ספציפית והוא: שהוא זהה בדיוק. **דוגמה 2.8.** נראה בדוגמה ממקודם:

```

1 2000 sum: add a0, a0, a1
2 2004 ret

```

3 המחסנית

לפעמים נרצה לאחסן דברים נוספים אשר לא יתאימו ברגיסטרים. לשם כך, נוכל להשתמש במחסנית, אשר פועלת בתצורת *LIFO*. על מנת להשתמש במחסנית, נדרש מצביע אליה כדי לגשת אליה. לכן, יש לנו את הרגיסטר בשם *sp* אשר **תמיד** מצביע לנקודה הכי נמוכה במחסנית (כאשר המחסנית גדלה למטה □ כאשר מכניסים איבר מכניסים אותו ללמטה).

דוגמה 3.1. נרצה לממש את הפונקציה הבאה באסמבלי.

```

1 int leaf(int g, int h, int i, int j){
2 }
3     int f;
4     f = g + h - i + j;
5     return f;
6 {

```

נניח ואנו משום מה חייבים להשתמש ב- s_0, s_1 שכפי שאנו יודעים הם רגיסטרים שחייבים לשמור על הערך שלהם.
נרצה לראות כיצד מגבים אותם:

```

1 Leaf:
2     addi sp, sp, -8    # adjust stack for 2 items
3     sw s1, 4)sp(      # save s1 for use afterwards
4     sw s0, 0)sp(      # save s0 for use afterwards
5     add s0, a0, a1     # s0 = g + h
6     add s1, a2, a3     # s1 = i + j
7     sub a0, s0, s1     # return value )g + h( - )i + j(
8     lw s0, 0)sp(      # restore register s0 for caller
9     lw s1, 4)sp(      # restore register s1 for caller
10    addi sp, sp, 8     # adjust stack to delete 2 items
11    ret                # returning to original location

```

3.1 קריאה לפונקציה מתוך פונקציה

ראינו כי אנו משתמשים ברגיסטר ra לכתובת החזרה, אך אם אנו קוראים לפונקציה בתוך פונקציה, אנו צריכים לשמור שני ערכים עבור כתובת ההחזרה: כתובת ההחזרה מהפונקציה המקוננת אל הפונקציה וכתובת ההחזרה מהפונקציה אל המקור.
נעשה זאת באמצעות שמירה של ra למחסנית כפי שראינו עכשיו.

דוגמה 3.2

```

1 int sumSquare(int x, int y(
2 }
3     return mult)x,x( + y;
4 {

```

נממש את הקוד הבא באסמבלי כך:

```

1 sumSquare: # param x in a0, y in a1, return in a0
2     addi sp, sp, -8    # space on stack
3     sw ra, 4)sp(      # save ret addr
4     sw a1, 0)sp(      # save y )since we are going to use a1(
5
6     mv a1, a0
7     jal mult          # call mult: get param in a0,a1; return
                        # in a0
8     lw a1, 0)sp(      # restore y
9     add a0, a0, a1
10

```

```
11  lw ra, 4)sp(      # get ret addr
12  addi sp, sp, 8    # restore stack
13  ret              # = jr ra
```